

Young Innovators · Week 11

Wi-Fi Automatic Plant Watering System

Build a smart farm that waters itself and reports to your phone



Ages 12-14

ESP32 + Arduino IDE

IoT

Did you know?

Over 90% of the water Pakistan uses goes to farming. Smart watering helps save it.



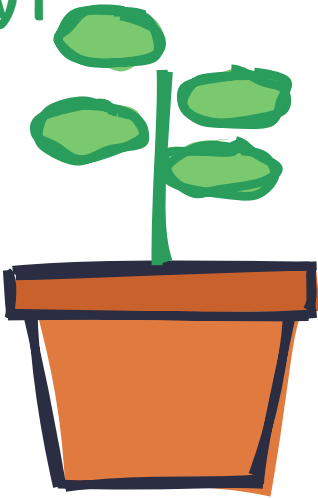


Picture this...

Your family goes on holiday for a week

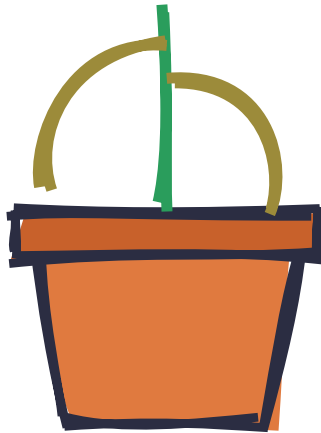


Day 1



Happy, healthy plant

Day 7



Nobody watered it...

Now think bigger

A farmer in Punjab floods the whole field on a schedule, even when the soil is already wet. Water is wasted and crops can rot.



Can a machine decide when a plant really needs water?

In real life

Smart irrigation controllers in farms and parks water only when sensors say the soil is dry.





Learning aims

By the end of this session, I can...

1  Explain how a **soil moisture sensor** and a **DHT22** measure the environment

2  Control a **pump** safely through a **relay**

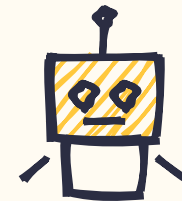
3  Program an **ESP32** to decide using sensor thresholds

4  Host a live **status webpage** over Wi-Fi

5  Evaluate how much **water** the system could save

Keywords

IoT · ESP32 · analog input · threshold ·
relay · web server · IP address ·
calibration



Watch and think

Video time!



The Internet: Wires, Cables & Wi-Fi

Code.org · about 7 minutes · youtu.be/ZhEf7e4kopM

Watch for...

- 1 How is information sent as **bits**?
- 2 How does Wi-Fi send bits without wires?
- 3 What is **bandwidth**?

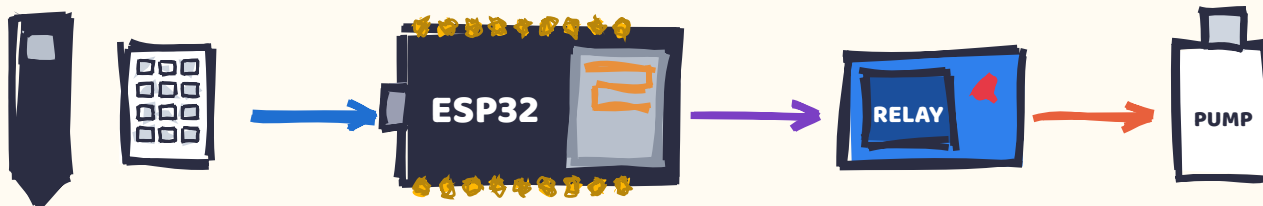
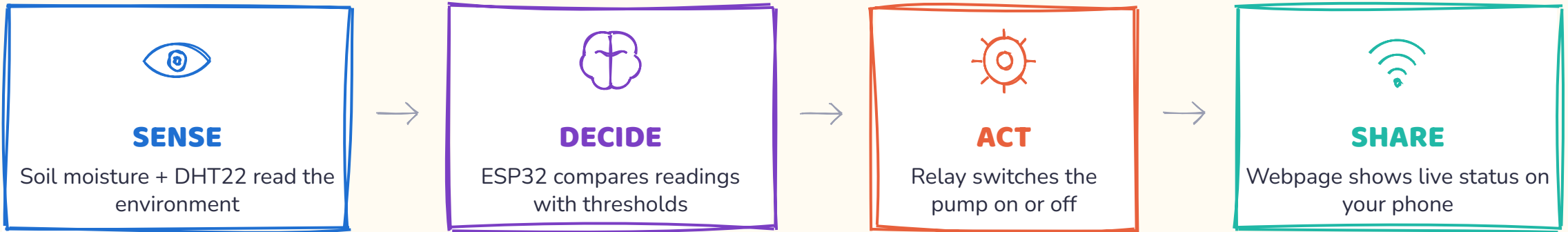


After the video: Our plant sends its soil reading over Wi-Fi. Draw the path from the plant to your phone.



The big picture

Sense, decide, act, share



Wi-Fi to phone

In real life

This is the Internet of Things (IoT): everyday objects that sense, decide and talk over the internet.





Meet the brain

The ESP32: a Wi-Fi microcontroller



Careful: ESP32 pins use **3.3 V**. Never connect a 5 V signal straight to them.

	Arduino Uno	ESP32
Wi-Fi + Bluetooth	No	Yes
Logic voltage	5 V	3.3 V
Processor	16 MHz, 1 core	240 MHz, 2 cores
Analog pins	6	18



In real life

ESP32 chips are inside many smart plugs, smart bulbs and video doorbells.





Sensor 1

Capacitive soil moisture sensor



How it works

Water changes how the sensor stores electric charge (its **capacitance**). The ESP32 reads this as a number from **0 to 4095**.

Wetter soil = lower number.

Calibrate it!

Soil	Yours	Example
Dry (air)	_____	~3000
Damp	_____	~2200
In water	_____	~1300

Why capacitive?

Cheap fork sensors pass current through the soil and corrode in weeks. Capacitive sensors last much longer.

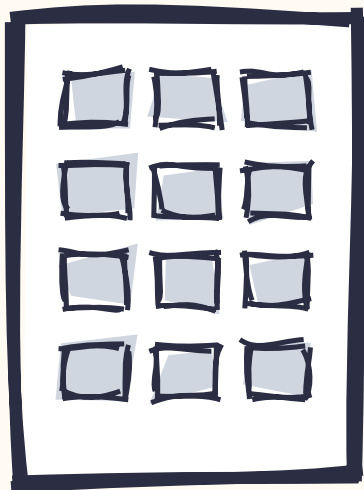
In real life

Farm sensors report soil moisture at different depths across a whole field.



Sensor 2

DHT22: temperature and humidity



What it measures

Temperature: -40 to 80 °C

Humidity: 0 to 100 % (water vapour in the air)

Read it at most once every **2 seconds**.

Why the plant cares

Hot, dry air: water evaporates fast, the plant needs water sooner.

Very humid (for example before rain): we can skip watering and save water.



Smart rule

Water only if the soil is dry AND the air is not very humid.

In real life

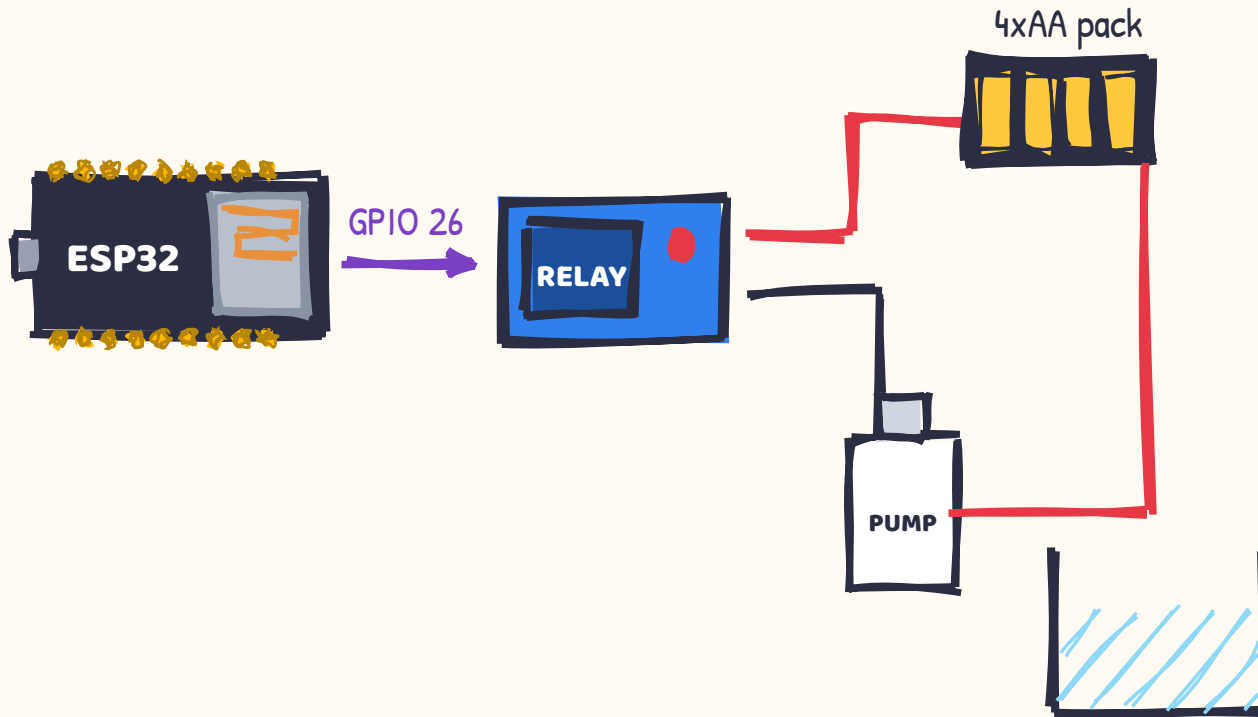
Greenhouses use the same sensors to control fans, misters and vents automatically.





Act safely

Why we need a relay to run the pump



The problem

An ESP32 pin gives only a tiny current (about 40 mA max). A pump needs much more and would damage the pin.

The solution

A **relay** is a switch flipped by a small signal. The pump gets its power from its own **battery pack**.

Safety: low voltage only, keep the water tank below and away from the electronics.



Build it

Wiring guide

Part	Pin	Connect to
Soil sensor	AOUT	GPIO 34 (analog)
Soil sensor	VCC / GND	3.3 V / GND
DHT22	DATA	GPIO 4
DHT22	VCC / GND	3.3 V / GND
Relay module	IN	GPIO 26
Relay module	VCC / GND	VIN (5 V) / GND
Pump	+ / -	Relay COM-NO + 4xAA pack

Pin tips

Use **GPIO 32-39** for analog sensors: the ADC2 pins stop working when Wi-Fi is on.

Some relay modules need a **5 V** supply but accept a 3.3 V signal. Check yours.

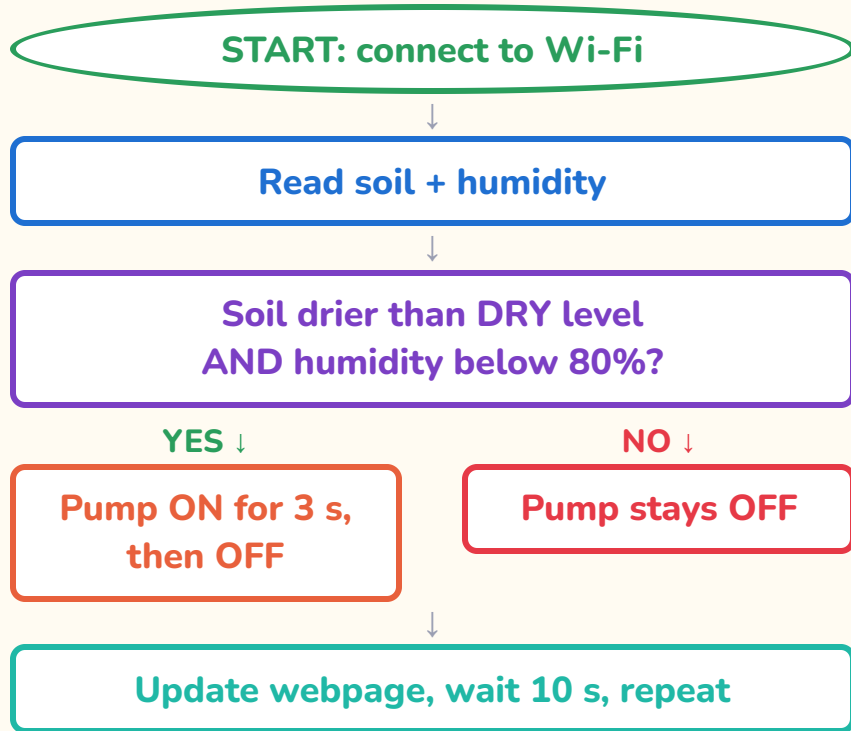
Test first

Build the ESP32 + DHT22 part in Wokwi before touching real parts.



Decide

The decision logic (flowchart)



Why short bursts?

Water takes time to soak in. If the pump runs until the sensor reads wet, it overwaters. Water a little, wait, then check again.



Pro idea: hysteresis

Use two levels: start watering below 30 %, stop above 45 %. This stops the pump flicking on and off.

In real life
Home thermostats use the same two-level trick.



Code it

The ESP32 program (Arduino IDE)



```
#include <WiFi.h>
#include <WebServer.h>
#include "DHT.h"
#define SOIL 34           // analog pin (ADC1)
#define RELAY 26
DHT dht(4, DHT22);
WebServer server(80);
int DRY = 2800;           // from YOUR calibration

void setup() {
  Serial.begin(115200);
  pinMode(RELAY, OUTPUT);
  dht.begin();
  WiFi.begin("SchoolWiFi", "password");
  while (WiFi.status() != WL_CONNECTED) delay(500);
  Serial.println(WiFi.localIP()); // open this in a browser
  server.on("/", showPage); // showPage() builds the HTML
  server.begin();
}

void loop() {
  int soil = analogRead(SOIL);
  float hum = dht.readHumidity();
  if (soil > DRY && hum < 80) {
    digitalWrite(RELAY, HIGH); delay(3000); // water 3 s
    digitalWrite(RELAY, LOW);
  }
  server.handleClient();
}
```

Read the code

WiFi.begin joins the network.
analogRead gets the soil number.
if checks both conditions.
digitalWrite flips the relay.
server answers your phone.



Heads up

Many relay modules are active LOW. If the pump runs when it should be off, swap HIGH and LOW.





Share

Your plant's live status webpage



My Plant

Soil moisture 42 % (OK)

Temperature 31 °C

Humidity 40 %

Pump OFF

Last watered 08:15

[Water now](#)<http://192.168.1.45>

How your phone sees it

1. The ESP32 joins the school Wi-Fi and gets an **IP address**.
2. Your phone, on the same Wi-Fi, opens that address.
3. The ESP32 replies with a small **HTML page** built from live readings.



In real life

Smart home apps talk to devices in exactly this way: a request and a reply.

Challenge

Add a 'Water now' button that calls /water to run the pump for 3 seconds.





Your turn

Build missions



1. Simulate

Build ESP32 + DHT22 in Wokwi. Use a potentiometer to fake the soil sensor.



2. Calibrate

Record dry, damp and wet readings. Choose your DRY level.

Team roles

Wiring lead • Coder • Tester • Reporter.
Swap roles halfway!



3. Wire

Connect sensors, relay and pump. Teacher checks before power-on.



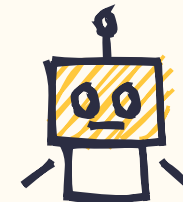
4. Upload & connect

Upload, find the IP in Serial Monitor, open it on a phone.



5. Test

Dry the soil, watch it water. Measure how many ml it uses.





Test and debug

Bug hunter's guide

**Soil reading is always 0 or 4095**

Wrong pin, or using an ADC2 pin with Wi-Fi on. Use GPIO 34.

**ESP32 will not connect to Wi-Fi**

Check spelling of name and password. ESP32 only works on 2.4 GHz networks.

**DHT22 shows 'nan'**

Check the data wire, and wait 2 seconds between readings.

**Pump never turns on**

Relay trigger level (HIGH/LOW), pump battery empty, or COM/NO wired wrong.

**Page will not open on phone**

Phone must be on the same Wi-Fi; type the exact IP address.

Engineer's habit

Change one thing at a time, then test again.





Why it matters

From our classroom plant to real farms



Real-world impact

Farmers can save water and electricity by watering only when needed.

Remote farms can be checked from a phone.

Data over weeks shows the best time to water.

UN Global Goals this project supports:

2

Zero Hunger

6

Clean Water

13

Climate Action



Think about it

If our school garden has 20 plants, and smart watering saves 1 litre per plant per day, how much water is saved in a month?

Coming up

YI-12: build a phone app for this system. YI-16: 3D-print its protective case.





Show what you know

Recap and exit ticket

Exit ticket

- ☒ Why does the pump need a relay?
- ☒ Why must the soil sensor use GPIO 34 (ADC1) with Wi-Fi on?
- ☒ What is the IP address used for?
- ☒ How does the humidity check save water?



Extension challenge

Log readings every 10 minutes and show a mini graph on the webpage, or send an alert when the water tank is empty.



Well done!

You built a real IoT device that senses, decides, acts and shares.

